

MATA58

SISTEMAS OPERACIONAIS

Duas aulas guiadas teórico-práticas

Processos, IPC, concorrência e sincronização

Formato

2 aulas de 1h30 cada, com roteiro minuto a minuto, demonstrações em C, atividades para os alunos e gabarito para o aplicador.

Arquivos-base

fork2.c | pipe2.c | peterson-code.c | prod_cons.c

Professor: Maycon Leone M. Peixoto

Preparação do laboratório e comandos validados

Antes das aulas, deixe os quatro arquivos em uma mesma pasta Linux e confirme a presença do GCC. Os comandos abaixo foram revisados considerando as bibliotecas efetivamente usadas pelos exemplos.

1. fork2.c

```
gcc -Wall -Wextra fork2.c -o fork2
./fork2
```

Não precisa de biblioteca adicional para fork(), malloc() ou getpid().

2. pipe2.c - usa sqrt()

```
gcc -Wall -Wextra pipe2.c -o pipe2 -lm
./pipe2
```

Importante

O parâmetro -lm é necessário para linkar a biblioteca matemática usada por sqrt(). Sem ele, o linker retorna “undefined reference to sqrt”.

3. peterson-code.c - usa pthreads e sleep()

Antes de compilar, acrescente ao início do arquivo:

```
#include <unistd.h>
```

Depois, compile com:

```
gcc -Wall -Wextra peterson-code.c -o peterson -pthread
./peterson
```

Atenção ao exemplo original

O arquivo usa sleep(), portanto precisa de <unistd.h>. Além disso, char cr[8] é pequeno para a string "thread1#####". Para uma demonstração segura, aumente para char cr[16] ou substitua a string por "thread1".

4. prod_cons.c - pthreads e semáforos POSIX

Para deixar o código corretamente declarado, recomenda-se também acrescentar:

```
#include <time.h>
```

Compile com:

```
gcc -Wall -Wextra prod_cons.c -o prod_cons -pthread
./prod_cons
```

Nota de portabilidade

Em Linux/glibc atual, -pthread é suficiente para pthreads e semáforos POSIX usados neste exemplo. Em ambientes Linux muito antigos, sem_* podia exigir bibliotecas adicionais; para o laboratório atual, prefira o comando acima.

Aula 1 - Processos, fork() e comunicação com pipe()

Duração: 1h30 | Modalidade: teórico-prática | Arquivos: fork2.c e pipe2.c

Objetivo da aula

Compreender criação de processos, retorno de fork(), independência dos espaços de memória e a necessidade de mecanismos explícitos de IPC. Ao final, o aluno implementará uma comunicação simples entre pai e filho por pipe.

Roteiro de 90 minutos

Tempo	Etapa	Condução
0-10 min	Revisão	Programa x processo; PID; espaço de memória.
10-30 min	fork()	Previsão da árvore de processos usando fork2.c.
30-42 min	Demonstração	Compilar, executar e discutir memória após fork().
42-52 min	Atividade 1	Questões curtas sobre processo, PID e memória.
52-67 min	IPC e pipe()	Conceito de pipe, fd[0]/fd[1] e leitura/escrita.
67-80 min	Demonstração pipe2.c	Mapear produtores, pipes e processo de cálculo.
80-88 min	Prática	Completar um programa pai-filho usando pipe.
88-90 min	Fechamento	Síntese e entrega das respostas.

0-10 min - Abertura

Pergunta de abertura ao quadro: “Programa e processo são a mesma coisa?” Espere respostas antes de definir.

Explique: programa é o conjunto de instruções armazenado; processo é uma instância desse programa em execução, com PID, estado, registradores, pilha, heap e demais recursos associados.

Perguntas rápidas para a turma

- 1) Dois processos executando o mesmo programa têm necessariamente o mesmo PID?
- 2) Dois processos diferentes têm a mesma pilha?
- 3) Um processo pode criar outro processo?

10-30 min - Entendendo fork()

Mostre apenas o trecho principal de fork2.c e peça uma previsão antes de executar:

```
for (i = 0; i < tamanho; i++) {
    pid = fork();
    if (pid == 0) {
        v[i] = i + 5;
    }
}
```

Explique que fork() duplica o processo corrente. No processo filho retorna 0; no pai retorna o PID do filho. Como todos os processos continuam o laço, três chamadas sucessivas podem levar a $2^3 = 8$ processos.

Pergunta-chave

“Sem executar: quantos processos existirão ao final das três iterações? Desenhe a evolução 1 -> 2 -> 4 -> 8.”

30-42 min - Demonstração: memória depois do fork()

```
gcc -Wall -Wextra fork2.c -o fork2
./fork2
```

Chame atenção para o vetor alocado antes do fork(). O fato de ter sido criado com malloc() não o torna memória compartilhada entre processos. Após o fork, pai e filhos possuem espaços de memória virtual independentes.

Ideia a fixar

Se um filho faz $v[i] = i + 5$, ele altera a sua cópia do espaço de memória. O processo pai não passa a enxergar automaticamente esse valor.

42-52 min - Atividade 1: resposta

1. Por que usar malloc() antes do fork() não transforma o vetor em memória compartilhada?

2. Depois de fork(), pai e filho continuam executando a partir de onde?

3. Qual valor de retorno de fork() permite identificar o processo filho?

4. O pai necessariamente enxergará os valores que os filhos escreverem no vetor? Justifique.

5. Cite um mecanismo que poderia ser usado para enviar explicitamente um resultado do filho ao pai.

52-67 min - IPC e pipe()

Faça a transição: se processos têm memória independente, a cooperação entre eles exige mecanismos de IPC (Interprocess Communication). Um pipe fornece um fluxo de bytes com duas extremidades:

```
int fd[2];
pipe(fd);

fd[0] // leitura
fd[1] // escrita
```

Regra para memorizar

fd[0] lê; fd[1] escreve. Em cada processo, feche a extremidade que não será usada.

67-80 min - Leitura guiada de pipe2.c

O exemplo cria cinco pipes, depois processos que produzem componentes do cálculo e um processo que recebe valores pelos pipes e calcula as raízes. O foco não é a fórmula de Bhaskara, mas o fluxo de dados entre processos.

```
Processo de a ---- 2a ----\
Processo de b ---- -b -----> processo de cálculo ---- x1/x2 ----> pai
Processo delta ---- delta --/
```

Mostre chamadas close(), write() e read(). Pergunte em cada trecho: “quem escreve?”, “quem lê?” e “qual extremidade deve ser fechada?”.

```
gcc -Wall -Wextra pipe2.c -o pipe2 -lm
./pipe2
```

80-88 min - Atividade prática

Complete o programa para que o filho calcule $10 + 20$, escreva o resultado em um pipe e o pai leia e imprima o valor.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int fd[2];
    int valor;
    pid_t pid;

    pipe(fd);
    pid = fork();

    if (pid == 0) {
        /* FILHO: completar */
    } else {
        /* PAI: completar */
    }

    return 0;
}
```

Peça explicitamente que usem `close()`, `write()` e `read()`, e que expliquem por que uma variável comum não bastaria.

88-90 min - Fechamento

Resumo em uma frase

`fork()` cria outro processo; processos têm memória independente; para trocar dados explicitamente, usamos mecanismos de IPC como `pipe()`.

Aula 1 - Gabarito do aplicador

Atividade 1

- 1. Porque malloc() aloca memória no espaço virtual do processo; após fork(), pai e filho passam a ter espaços de endereçamento logicamente independentes.
- 2. Ambos continuam a partir da instrução seguinte à chamada de fork().
- 3. No filho, fork() retorna 0.
- 4. Não. Alterações em memória comum de um processo não são automaticamente propagadas ao outro processo.
- 5. Pipe, memória compartilhada, fila de mensagens, socket, entre outros.

Solução da prática

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int fd[2];
    int valor;
    pid_t pid;

    if (pipe(fd) == -1) return 1;
    pid = fork();
    if (pid == -1) return 1;

    if (pid == 0) {
        close(fd[0]);
        valor = 10 + 20;
        write(fd[1], &valor, sizeof(valor));
        close(fd[1]);
    } else {
        close(fd[1]);
        read(fd[0], &valor, sizeof(valor));
        printf("Resultado recebido: %d\n", valor);
        close(fd[0]);
    }

    return 0;
}
```

Se sobrar tempo

Pergunte o que acontece se o pai executar read() enquanto ainda não há dados no pipe. Direção da resposta: uma leitura bloqueante normalmente espera até haver dados ou até que as extremidades de escrita sejam fechadas.

Aula 2 - Concorrência, semáforos e monitores

Duração: 1h30 | Modalidade: teórico-prática | Arquivos: peterson-code.c, prod_cons.c e exemplos Java da pasta Monitor

Objetivo da aula

Compreender condições de corrida e região crítica, revisar exclusão mútua com Peterson e semáforos e introduzir o conceito de monitor em Java com synchronized, wait() e notify().

Roteiro de 90 minutos

Tempo	Etapas	Condução
0-10 min	Threads e race condition	Memória compartilhada e interleavings.
10-22 min	Região crítica + Peterson	Exclusão mútua e espera ocupada.
22-38 min	Semáforos	wait/post; mutex, empty e full.
38-52 min	Produtor-consumidor	Ler o código e simular os semáforos.
52-62 min	Por que monitores?	Do semáforo explícito a uma abstração de alto nível.
62-75 min	Mailbox sem sincronização	Executar a versão 3b e identificar a condição de corrida.
75-84 min	synchronized + wait/notify	Executar a versão 3s e comparar.
84-90 min	Atividade final	Responder questões e fechar conceitos.

0-10 min - Threads e condição de corrida

Pergunta de abertura: “Na aula anterior vimos que processos têm memória independente. E threads do mesmo processo?”

Threads do mesmo processo compartilham código, dados, heap e recursos. Cada thread mantém sua própria pilha e contexto de execução. O compartilhamento facilita a comunicação, mas permite acessos simultâneos ao mesmo estado.

```
int contador = 0;

// duas threads executam:
contador++;
```

Mostre que o incremento pode ser decomposto conceitualmente em ler, incrementar e escrever. Intercalações diferentes podem produzir resultados diferentes: isso é uma condição de corrida.

10-22 min - Região crítica, exclusão mútua e Peterson

Região crítica é o trecho que acessa um recurso compartilhado e que precisa ser protegido quando a execução concorrente pode gerar inconsistência. Exclusão mútua significa permitir apenas uma thread por vez nessa região.

No exemplo de Peterson, mostre turn e interested[]. O algoritmo é didático para duas threads e evidencia a ideia de entrada, região crítica e saída.

```
gcc -Wall -Wextra peterson-code.c -o peterson -pthread
./peterson
```

Antes de compilar

Acrescente `#include <unistd.h>` para `sleep()` e aumente `char cr[8]` para pelo menos `char cr[16]` ou reduza a string armazenada.

```
interested[thread] = TRUE;
turn = thread;
while (turn == thread && interested[other] == TRUE)
    ;

/* região crítica */

interested[thread] = FALSE;
```

Pergunte: o que a thread faz enquanto fica presa no while? Introduza espera ocupada (busy waiting): ela continua consumindo CPU enquanto espera.

22-38 min - Semáforos

Introduza as operações wait/down/P e post/up/V. No exemplo produtor-consumidor há três semáforos:

```
sem_t mutex, empty, full;

sem_init(&mutex, 0, 1);
sem_init(&empty, 0, N);
sem_init(&full, 0, 0);
```

- mutex=1: a região crítica começa livre.
- empty=N: todas as posições do buffer começam livres.

- full=0: ainda não existe item disponível para consumo.

Pergunta para a turma

Qual é a diferença entre mutex e empty/full? mutex protege o acesso ao buffer; empty/full representam estados de disponibilidade.

38-52 min - Produtor-consumidor

Recomenda-se acrescentar `#include <time.h>` ao `prod_cons.c`. Depois compile com pthreads:

```
gcc -Wall -Wextra prod_cons.c -o prod_cons -pthread
./prod_cons
```

Produtor:

```
sem_wait(&empty);
sem_wait(&mutex);
insert_item(item);
sem_post(&mutex);
sem_post(&full);
```

Consumidor:

```
sem_wait(&full);
sem_wait(&mutex);
item = remove_item();
sem_post(&mutex);
sem_post(&empty);
```

Atividade rápida: para $N=5$, comece com `empty=5` e `full=0`. Após dois itens produzidos e um consumido, peça os valores. Resposta: `empty=4` e `full=1`. Reforce que `empty + full = N`.

52-62 min - Por que estudar monitores?

Faça a transição: semáforos funcionam, mas deixam a disciplina de sincronização muito exposta ao programador. É fácil esquecer uma operação, inverter a ordem ou proteger o recurso errado. Monitores oferecem uma abstração de nível mais alto: o estado compartilhado e as operações sincronizadas ficam encapsulados no mesmo objeto.

Ideia central

Um monitor combina dados compartilhados, operações que os acessam e exclusão mútua. Em Java, todo objeto pode servir de base para sincronização por meio de `synchronized`.

Use a analogia do material dos monitores: uma chave única para um banheiro compartilhado. Quem possui a chave entra; os demais esperam. A chave representa o lock associado ao objeto.

62-75 min - Mailbox sem sincronização: observar o problema

Abra a pasta Monitor/1. O exemplo cria uma única Mailbox3b compartilhada por um consumidor e dois produtores. A própria documentação do código afirma que essa versão não possui coordenação e pode misturar mensagens.

```
cd Monitor/1
javac Mailbox3b.java Producer3b.java Consumer3b.java ThreadSync3b.java
java ThreadSync3b
```

Em Mailbox3b.storeMessage(), os produtores escrevem caractere por caractere no mesmo array message. Como não há exclusão mútua, um produtor pode começar a escrever e ser interrompido por outro, produzindo uma mensagem inconsistente.

1. Qual é o recurso compartilhado neste exemplo?

2. Onde está a região crítica no Mailbox3b?

3. Por que aumentar MAXPROCESSTIME tende a tornar o problema mais visível?

75-84 min - Monitor em Java: synchronized, wait() e notify()

Agora abra Monitor/3. Em Mailbox3s, os métodos storeMessage() e retrieveMessage() são synchronized. Assim, uma thread precisa adquirir o lock do objeto Mailbox3s antes de executar cada método sincronizado.

```
public synchronized void storeMessage(String msg) {
    ...
    youHaveMail = true;
    notify();
}

public synchronized String retrieveMessage() {
    while (youHaveMail == false) {
        wait();
    }
    ...
}
```

Explique: wait() faz a thread liberar o monitor e entrar em espera; notify() acorda uma das threads que aguardam naquele monitor. Esses métodos devem ser usados em contexto sincronizado.

```
cd Monitor/3
javac Mailbox3s.java Producer3s.java Consumer3s.java ThreadSync3s.java
java ThreadSync3s
```

Comparação a pedir à turma

Versão 3b: acesso concorrente sem coordenação. Versão 3s: synchronized protege o estado compartilhado e wait()/notify() evitam que o consumidor fique apenas consultando repetidamente a

caixa vazia.

84-90 min - Atividade final

4. O que um monitor acrescenta em relação à ideia de apenas compartilhar um objeto?

5. Qual é o papel de `synchronized` no exemplo `Mailbox3s`?

6. Por que `retrieveMessage()` usa `while` antes de `wait()` em vez de simplesmente continuar?

7. Qual a diferença conceitual entre espera ocupada e `wait()`?

Resumo em uma frase

Threads compartilham estado; condições de corrida exigem sincronização; semáforos são primitivas explícitas; monitores encapsulam o estado compartilhado e a sincronização, e `wait()/notify()` coordenam condições entre threads.

Aula 2 - Gabarito do aplicador

Atividade sobre a Mailbox

- 1. O objeto Mailbox3b, especialmente o array message e a variável youHaveMail, é compartilhado pelos produtores e pelo consumidor.
- 2. As operações que escrevem e leem o estado da mailbox são as regiões críticas; em especial, a escrita do array message em storeMessage() pode ser intercalada entre produtores.
- 3. Porque o atraso entre escritas aumenta a janela em que outra thread pode ser escalonada e interferir na mensagem que ainda está sendo montada.
- 4. O monitor associa exclusão mútua às operações sobre o estado compartilhado, encapsulando dados e sincronização no mesmo objeto.
- 5. synchronized exige que a thread adquira o lock/monitor do objeto antes de executar o método; enquanto uma thread está dentro de um método sincronizado do objeto, outra não executa simultaneamente outro trecho protegido pelo mesmo monitor.
- 6. A condição deve ser novamente verificada quando a thread acorda. O while expressa que a thread só pode prosseguir quando a condição desejada realmente for satisfeita.
- 7. Na espera ocupada, a thread continua executando testes e consumindo CPU. Com wait(), ela entra em estado de espera e libera o monitor até ser notificada.

Semáforos x monitores - síntese

Aspecto	Semáforo	Monitor
Nível	Primitiva explícita de sincronização	Abstração de nível mais alto
Exclusão mútua	Programador organiza wait/post	Associada às operações sincronizadas do objeto
Espera por condição	Modelada com semáforos adicionais	wait()/notify()/notifyAll()
Risco didático	Ordem incorreta das operações	Ainda exige uso correto das condições e locks

Comandos Java validados

```
# versão sem sincronização
cd Monitor/1
javac Mailbox3b.java Producer3b.java Consumer3b.java ThreadSync3b.java
java ThreadSync3b

# versão com monitor + wait/notify
cd ../3
javac Mailbox3s.java Producer3s.java Consumer3s.java ThreadSync3s.java
java ThreadSync3s
```

Observação para o estagiário

Os programas Java executam em laços infinitos. Interrompa cada demonstração com Ctrl+C depois de

observar algumas linhas de saída.

Checklist rápido para o estagiário

Antes da Aula 1

- Testar gcc e os arquivos fork2.c e pipe2.c.
- Confirmar que pipe2.c foi compilado com -lm.
- Abrir um terminal em fonte grande para demonstração.
- Não executar fork2.c antes de pedir a previsão da quantidade de processos.

Antes da Aula 2

- Adicionar #include <unistd.h> ao peterson-code.c.
- Corrigir o tamanho de cr no exemplo de Peterson (char cr[16]) ou encurtar a string.
- Adicionar #include <time.h> ao prod_cons.c.
- Compilar os exemplos C com -pthread.
- Confirmar que javac e java estão disponíveis.
- Deixar abertas as pastas Monitor/1 e Monitor/3 para comparar as versões sem sincronização e com monitor.
- Lembrar que os exemplos Java são laços infinitos: interromper com Ctrl+C.

Estratégia de condução

Sempre seguir a sequência

1. Apresentar o problema.
2. Pedir uma previsão.
3. Executar o código.
4. Comparar previsão e resultado.
5. Introduzir o conceito de SO.
6. Fazer uma atividade curta.